Exercise No: 1	Using Resource-Based Policies to Secure an S3 Bucket
Date of Exercise	07.08.2026

AIM:

To understand and implement access control in Amazon Web Services (AWS) using AWS Identity and Access Management (IAM) and Amazon Simple Storage Service (Amazon S3). To examine identity-based and resource-based policies, analyse the permissions assigned to IAM users and groups, assume IAM roles to obtain different access privileges, and understand how S3 bucket policies interact with IAM policies to control access to AWS resources.

DESCRIPTION:

This lab focused on understanding access control in Amazon Web Services (AWS) using Identity and Access Management (IAM) and Amazon Simple Storage Service (Amazon S3). The permissions assigned to the preconfigured IAM user devuser through the DeveloperGroup group were examined to understand how identity-based policies control access to AWS services. Read-level and write-level access to Amazon EC2 and S3 resources was tested, including the ability to create buckets and upload objects.

The activities also involved assuming the preconfigured BucketsAccessRole to examine how IAM roles provide different access permissions through temporary credentials. The role's attached policies and trust relationship were reviewed, and access to objects in different S3 buckets was tested. Finally, the bucket policy associated with bucket2 was examined to understand how resource-based permissions interact with identity-based policies to determine access to AWS resources.

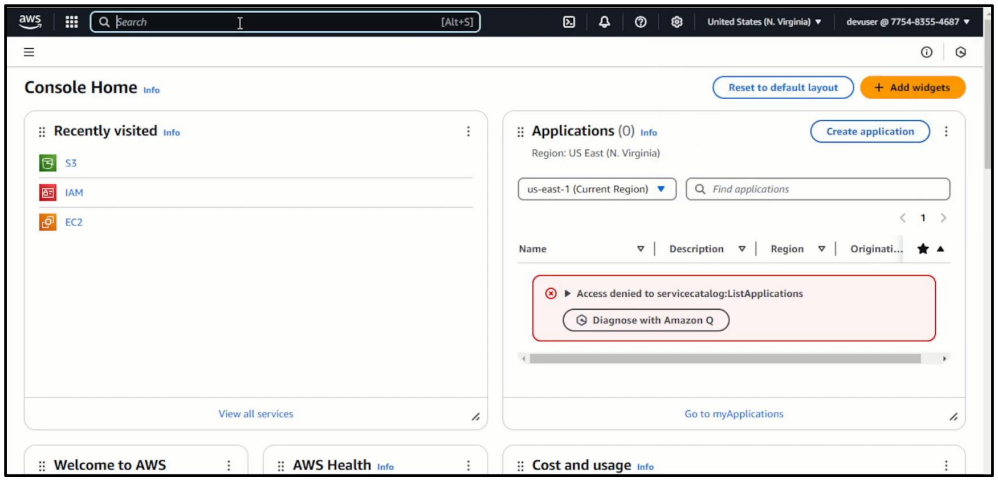
PROCEDURE:

Task 1: Accessing the Console as an IAM User

The AWS Management Console was accessed using the preconfigured IAM user named devuser. The login URL and credentials were obtained from the AWS Details panel of the lab environment. After successful authentication, the console was accessed without changing the default AWS Region. This established the IAM user's session for examining the permissions and access restrictions configured in the AWS environment.

Result: The AWS Management Console was successfully accessed using the devuser IAM account, providing access to AWS services according to the permissions assigned to the user.

OUTPUT:



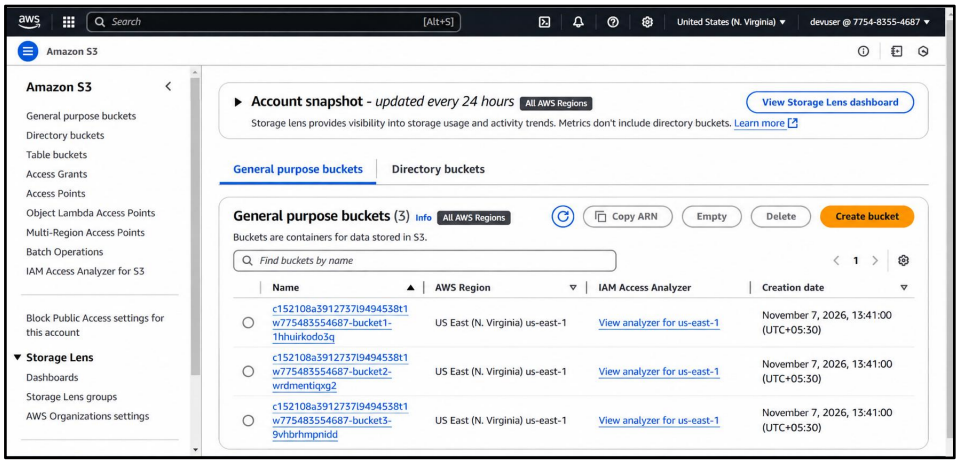
Task 2: Attempting Read-Level Access to AWS Services

In this task, the access permissions of the devuser IAM account were examined across different AWS services, including Amazon EC2, Amazon S3 and IAM. When accessing the EC2 Dashboard and attempting to view the Instances page, authorization errors were displayed, indicating that the user did not have sufficient permissions to perform these operations. Similarly, an attempt to access the EC2 key pair information while initiating the instance launch process was denied.

The Amazon S3 console was then accessed, where the three preconfigured buckets, bucket1, bucket2 and bucket3, were visible. However, the Access column displayed an insufficient permissions message for all three buckets, demonstrating the restrictions imposed by the assigned IAM permissions.

Result: The access restrictions of the devuser account were successfully examined. Although certain AWS service information was visible, access to specific EC2 operations and S3 bucket details was restricted due to insufficient IAM permissions.

OUTPUT:



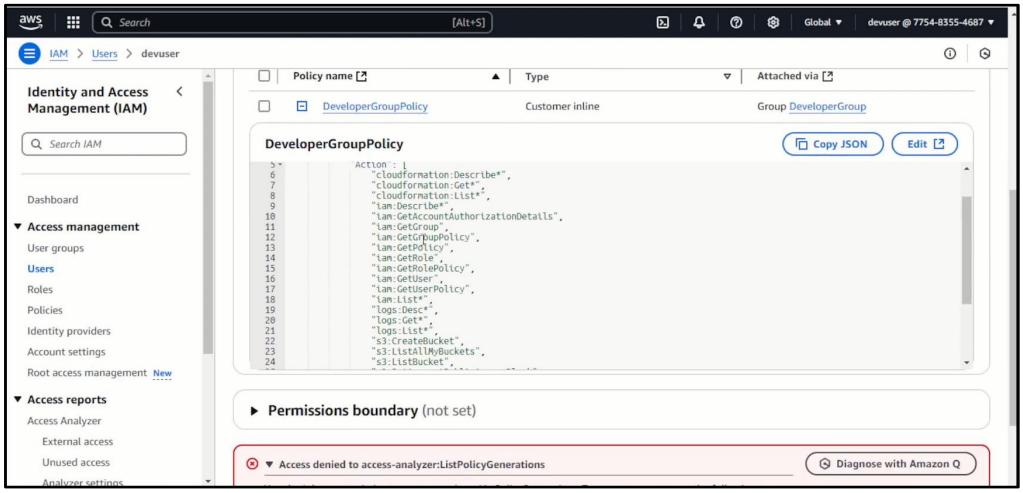
Task 3: Analysing the Identity-Based Policy Applied to the IAM User

In this task, the IAM configuration was examined to understand how permissions were assigned to the devuser account. The IAM console was accessed, and the DeveloperGroup group was selected to review its membership and attached permissions. It was observed that devuser was a member of this group and that the DeveloperGroupPolicy policy was attached to it.

The JSON policy document was examined to identify the permitted actions for different AWS services. The policy did not grant any Amazon EC2 actions, while it provided selected read-level IAM permissions and certain Amazon S3 bucket-related permissions. However, object-level S3 permissions were not included. The policy document was also copied and saved locally as DeveloperGroupPolicy.json for further analysis.

Result: The identity-based policy attached to the DeveloperGroup group was successfully analysed. It was confirmed that the permissions assigned to the group governed the access privileges of its members, restricting access to specific AWS services and operations.

OUTPUT:



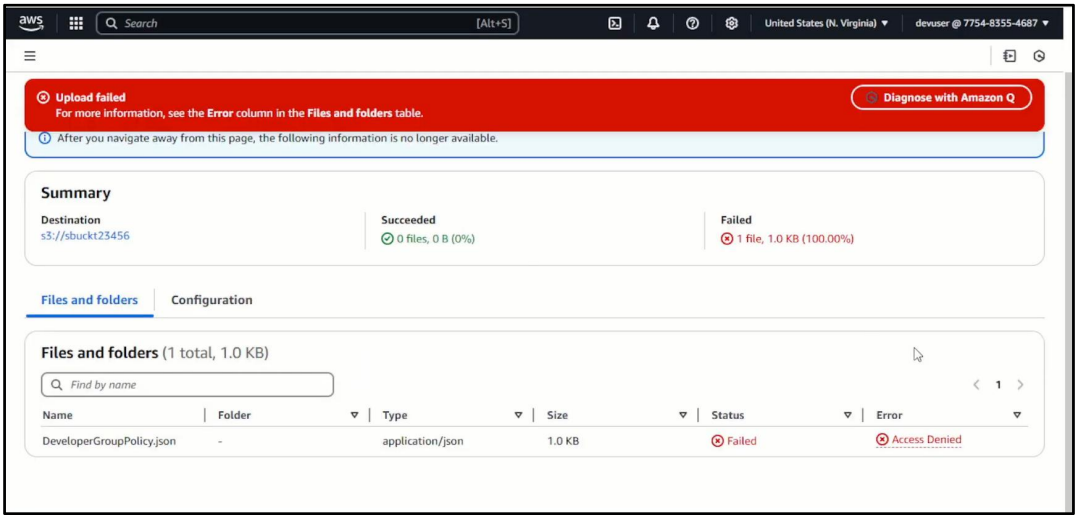
Task 4: Attempting Write-Level Access to AWS Services

In this task, write-level access to Amazon S3 was examined by attempting to create a new S3 bucket and upload an object. A new bucket was successfully created using a unique bucket name and the AWS Region US East (N. Virginia) (us-east-1). The previously saved DeveloperGroupPolicy.json file was then selected for upload to the newly created bucket. However, the upload operation failed with an Access Denied error, indicating that the IAM user did not have the required object-level permissions.

The attached DeveloperGroupPolicy was reviewed again to understand the difference between bucket-level and object-level permissions. It was identified that the policy allowed certain S3 bucket-related actions, including bucket creation, but did not grant the s3:PutObject permission required to upload files into a bucket.

Result: The difference between bucket-level and object-level permissions was successfully demonstrated. The IAM user was able to create an S3 bucket but was denied permission to upload objects because the required write-level permission was not included in the identity-based policy.

OUTPUT:



Task 5: Assuming an IAM Role and Reviewing a Resource-Based Policy

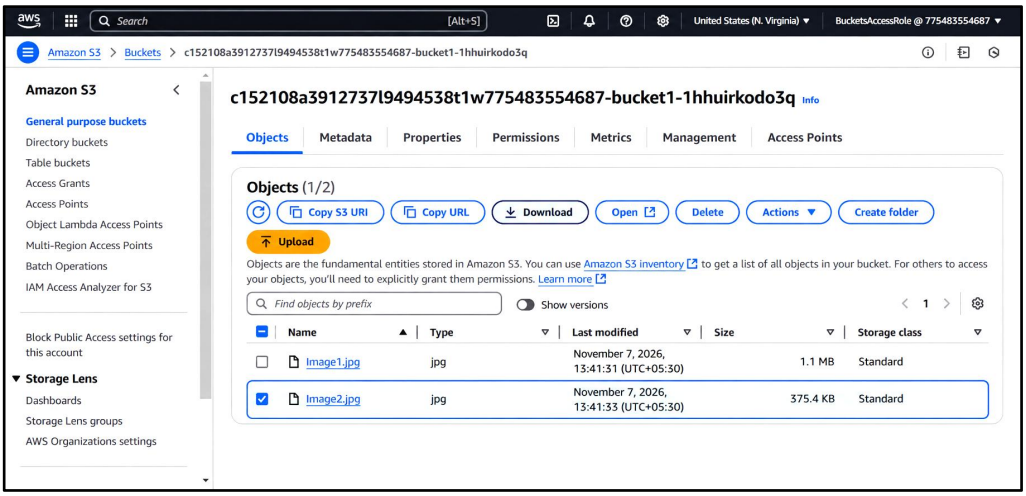
In this task, access to Amazon S3 resources was examined by switching between the devuser IAM account and the preconfigured BucketsAccessRole. Initially, attempts were made to download objects from bucket1 and bucket2 while logged in as devuser. Both operations resulted in Access Denied errors due to insufficient permissions.

The BucketsAccessRole IAM role was then assumed using the Switch Role option in the AWS Management Console. After successfully switching roles, an object named Image2.jpg was downloaded from bucket1. This confirmed that the assumed role had the required s3:GetObject permission. However, access to the IAM user groups page was restricted because the role did not have the iam:ListGroups permission.

The policies attached to the role were examined, including ListAllBucketsPolicy and GrantBucket1Access. It was observed that the role had permission to list S3 buckets and access objects within bucket1, but did not have the s3:PutObject permission through its identity-based policies. The trust relationship was also reviewed to verify that the devuser IAM account was authorised to assume the role. Finally, while assuming the role, the downloaded Image2.jpg file was successfully uploaded to bucket2, demonstrating the effect of additional permissions granted through the bucket's resource-based policy.

Result: The IAM role was successfully assumed, and access to S3 resources was verified. The experiment demonstrated how IAM roles provide temporary permissions and how their identity-based policies and bucket-level resource-based policies work together to determine access to AWS resources.

OUTPUT:



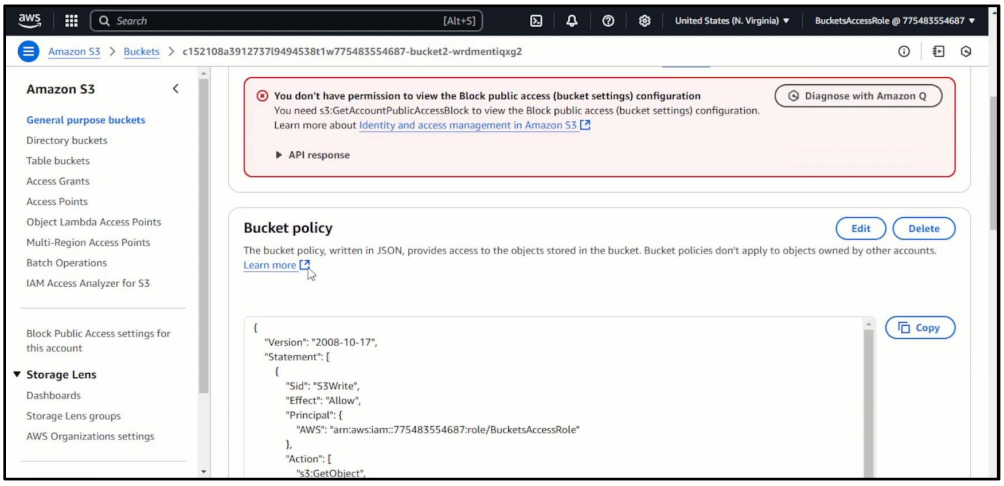
Task 6: Understanding Resource-Based Policies

In this task, the resource-based policy associated with bucket2 was examined to understand how permissions could be granted directly to an Amazon S3 bucket. The Permissions tab of the bucket was accessed, and the bucket policy was reviewed. It contained two statements, namely S3Write and ListBucket, which granted the BucketsAccessRole permission to perform s3:GetObject, s3:PutObject and s3:ListBucket actions on the specified bucket and its objects.

The interaction between the identity-based policies attached to the IAM role and the resource-based policy attached to bucket2 was analysed. It was confirmed that the successful upload to bucket2 was possible because its bucket policy explicitly granted the BucketsAccessRole permission to perform the s3:PutObject action, even though this permission was not provided by the role's identity-based policies.

Result: The resource-based bucket policy was successfully examined, demonstrating how permissions granted directly to an S3 bucket can work alongside identity-based IAM policies to control access to AWS resources.

OUTPUT:



BADGE:



PRESENTATION LINK:

[Insert YouTube / presentation link]

RESULT:

The experiment demonstrated AWS access control using IAM and Amazon S3. Identity-based policies were examined to understand the permissions assigned to IAM users and groups, and read-level and write-level access restrictions were tested. An IAM role was assumed to obtain temporary permissions, and S3 object access was verified. The interaction between identity-based policies and resource-based bucket policies was also demonstrated through object download and upload operations.